DRIVING SCHOOL BOOKING SYSTEM

By SER

Rayan, Ridaa, Sihaam, Eloise, Rahma
M01023728, M00994466, M01036513, M01002265, M01044772

Introduction

The aim of this project was to develop a full-stack Driving School Booking System that is accessible, efficient and easy to use for multiple user roles including Learners, Instructors and Admins. The system was designed to reflect real-world driving school operations by allowing each user type to interact with features relevant to their role.

Learners can select their preferred lesson type, either Manual or Automatic, choose an instructor and search for available lessons. They can book, reschedule and cancel lessons as well as manage their account details, change their password and delete their account when required. Instructors are provided with functionality to manage lesson schedules, view upcoming, past bookings also cancel them and control their availability. They can also search for students, remove learners from their list and update their account details.

The system includes an administrative role which provides oversight of both learners and instructors. Admin users can view all registered users, monitor activity levels, track lesson bookings and identify upcoming lessons. They are also able to detect inactive accounts, such as users with no activity for over 100 days and remove them when necessary. Admin users can also manage their own account settings, including changing their password.

All users are required to select their role before accessing the system, after which they can log in or register. Strong validation is implemented during registration and authentication, including checks for email format, password strength and phone number structure, ensuring data integrity and system security.

The application follows a full-stack architecture, integrating a Blazor frontend, an ASP.NET Web API backend and a SQL database managed through Entity Framework Core. This structure enables communication between components using RESTful APIs and supports scalability, maintainability and efficient data management.

This report is structured in accordance with the coursework brief. The design section outlines the user interface and user experience decisions alongside justification of the data structures and algorithms used. The testing section explains the testing approach and presents a table of test cases. Finally, the conclusion summarises the development process, highlights limitations and reflects on potential improvements.

Design

The design of the system focused on creating a clear and consistent interface across all user roles. A shared layout structure was applied to common pages, using the same fonts, colour scheme and styling throughout the application. This improves usability by allowing users to quickly become familiar with the interface. The navigation bar was reused across all roles, with slight variations in naming to reflect role-specific functionality.

The main variation in design occurs within the settings pages. These were intentionally designed differently for each user role, as learners, instructors and admins require access to different features and account controls. This ensures that users are only presented with relevant options, reducing complexity and improving clarity.

Selected Data Structures and Algorithms

Data Structures and Justification

The system implements custom linked lists to manage core application data, including Admins, Learners, Instructors and Lesson Bookings. Each linked list is constructed using a node-based structure, where each node stores a single object and a reference to the next node in the sequence.

This approach was chosen to meet the coursework requirement of implementing a custom data structure rather than using built-in collections such as lists or arrays. Linked lists provide a flexible and dynamic way to store data, as they do not require a fixed size and can grow or shrink as needed.

Separate linked lists were created for each entity type:

- Admin list
- Learner list
- Instructor list
- Lesson (booking) list

This separation ensures that each data structure focuses on a specific type of data, improving maintainability and organisation.

Linked lists are appropriate for this system because:

- Users and bookings are frequently added and removed

- The number of records is not fixed
- Sequential traversal is sufficient for searching by ID
- Memory is allocated dynamically rather than pre-defined

Algorithms and Analysis

The linked list structure supports several core algorithms used throughout the system. These algorithms are essential for supporting functionality such as user registration, booking management and account deletion.

1. Add Operation (Insert Node at End)

Pseudocode:

```
FUNCTION AddNode(data)
    CREATE newNode with data
    IF head is NULL
        head = newNode
    ELSE
        current = head
        WHILE current.next is not NULL
            current = current.next
        END WHILE
        current.next = newNode
    END IF
END FUNCTION
```

Analysis:

This algorithm traverses the list until it reaches the final node, then appends the new node. This ensures that new records are stored in the order they are added, which is suitable for managing bookings and user data.

2. Search Operation (Find by ID)

Pseudocode:

```
FUNCTION FindNode(id)
    current = head
    WHILE current is not NULL
        IF current.data.id == id
            RETURN current.data
        END IF
        current = current.next
    END WHILE
    RETURN NULL
END FUNCTION
```

Analysis:

This algorithm performs a sequential traversal of the linked list, checking each node until a match is found. This approach is sufficient for the system, as searching is based on unique identifiers such as booking ID or user ID.

3. Remove Operation (Delete Node)

Pseudocode:

```
FUNCTION RemoveNode(id)
```

```
    IF head is NULL
        RETURN false

    IF head.data.id == id
        head = head.next
        RETURN true

    current = head
    WHILE current.next is not NULL
        IF current.next.data.id == id
            current.next = current.next.next
            RETURN true
        END IF
        current = current.next
    END WHILE

    RETURN false
END FUNCTION
```

Analysis:

This algorithm removes a node by updating the reference of the previous node to skip the target node. This is used in features such as deleting accounts and removing bookings. It ensures that the structure of the linked list remains intact after deletion.

Application to the System

These algorithms directly support the functionality of the system. The add operation is used when creating new users or bookings, the search operation is used when retrieving specific records such as bookings or learners and the remove operation is used

when deleting accounts or cancelling lessons. Together, the data structure and algorithms provide an efficient and flexible way to manage dynamic data within the application.

Testing

While testing, two main areas were considered: The frontend usability testing and backend API testing. This was to ensure that the system worked as desired, handled errors where appropriate and the functional requirements were also met outlined in the project brief.

Frontend Testing:

The frontend was tested manually to verify that:

- Validation was set in place for logins and entering details
- Buttons triggering the correct actions e.g - leading to the correct page after logging in
- Booking flows being error free and intuitive
- Navigation bars working as expected – leading to the correct pages

Edge cases were also considered. Such as:

- Navigating back and forth between booking steps
- Submitting empty details

Backend API Testing:

The backend was testing using combination of **Manual API testing** using Postman, **Unit testing** for GET, POST, PUT DELETE endpoints and **Database validation** through the SQL Server Object Explorer. This was all done to ensure that:

- All endpoints returned the correct HTTP status
- Invalid requests would return the correct error messages
- Authentication and validation worked correctly where appropriate
- Migrations applied across different machines
- Database tables matched the relationships defined in the code

Integration testing was both tested once the frontend and backend were fully complete.

The full workflow included:

- Learner signing up -> Loggin in -> Booking a lesson
- Instructor receiving a request -> Accepting/Declining -> Viewing schedule

- Admin viewing user activity and database entries

TESTING TABLE:

TEST ID	FEATURE	TEST SCENARIO	INPUT PREDICTIONS	EXPECTED RESULT	FINAL RESULT	STATUS (PASS/FAIL)
T1	User Authentication	Learner signs up with valid details	Email, password, confirms password	Account is created successfully and is stored in database	Works as expected	PASS
T2	User Authentication	Learner signs up with invalid details	Invalid email format	Error messages displayed	Error displayed correctly	PASS
T3	Login	Instructor logs in with correct details	Valid email and password	Successfully logged in, redirects to instructor dashboard	Works as expected	PASS
T4	Login	Instructor logs in with incorrect details	Valid email and incorrect password	Error message displayed	Error displayed correctly	PASS
T5	Booking system	Learner books a lesson with an instructor	Selected date, time	Booking saved in database and visible in learner schedule	Booking Created successfully	PASS
T6	Booking Management	Learner cancels a booking	Booking ID	Booking removed from database	Booking cancelled successfully	PASS

T7	Booking Management	Learner reschedules a booking	Booking ID	Booking updated in database	Booking rescheduled successfully	PASS
T8	Instructor Dashboard	Instructor declines a learner request	Request ID	Request changes to declined	Status updated correctly	PASS
T9	Instructor Dashboard	Instructor declines a learner request	Request ID	Request status changes to declined	Status updated correctly	PASS
T10	Notifications	Instructor received notifications on of schedule changes	Learner reschedules booking	Notifications appear in instructor inbox	Notification received	PASS
T11	Admin Panel	Admin deletes inactive accounts	USER ID	Account removed from database	Account deleted successfully	PASS
T12	Admin Panel	Admin views all learner accounts	Database query	Table displays all learner accounts	Data displayed correctly	PASS
T13	API Testing	GET/api/bookings returns correct data	API call with valid token	Return lists of bookings in JSON format	Data returned correctly	PASS
T14	API Testing	POST/api/bookings with missing fields	Missing date/time	Returns bad request	Error returned correctly	PASS
T15	Security	Access API without JWT Token	No token provided	Returns 401 unauthorised	Unauthorised response returned	PASS

T16	Database	Apply EF core Migrations	Update	Database created with all tables	Migration applied successfully	PASS
T17	Database	Preventing duplicate Admin Table creations	Existing Admin table	Migration should not duplicate table	No duplicates created	PASS
T18	Responsiveness	Website on laptop view	Laptop view size	Layout adjusts accordingly	Responsive layout confirmed	PASS
T19	Error Handling	Invalid routes accessed	Wrong URL	Page not found displayed	Error Page displayed	PASS
T20	Performance	Load Instructor list	Database Containing many instructors	Page loaded at a fast pace	Loads correctly	PASS

Conclusion and Reflections

Group Conclusion

The main technical difficulty that the group had faced is the backend. This was due to errors through testing and merging as well as software corruption which was time consuming. Although the design process should be quicker than the backend code and testing, more time was spent on the design aspect. This should have been reversed. This would help to ensure the APIs and algorithms pass for the status and the result comes back as the Tester expected. Furthermore, errors could have been dealt with a bit more efficiently through time and communication as these slow progress down for the main Developers. Communication and time keeping could have been improved through the Leader and Secretary as the Leader should be able to open issues and suggestions up for discussion and provide time frames for each milestone as well as the Secretary being be able to write down progress milestones as well as reminding the group of the timeline.

Rayan Ali, M01023728 – Team Leader

Although everything came together in the end, I believe that the way the project was approached could be improved. Because this was a big project, there are some considerations I should have taken such as how long the frontend, backend and linking them both together would take. My focus was Creating the Instructor Controller, and the instructor frontend which made multiple conflicts within the git repository.

Models, Data Structures, dB Context and namespaces were misnamed, and I had used HTML instead of Razor as my VS was corrupted and refused to clone the team's repository. This made me create separate project folders and files which created these conflicts. Removing use cases for functionality design also took time. If I was to encounter an issue like this in future, I would give myself and the team better time frames for when each part should be finished and start with the backend first to test how the API responds. I would have also learned the difference between HTML and Razor much earlier whilst linking the backend and frontend.

Another thing that could be improved is making sure there is communication between team members. This is important for group work as it can leave members confused on what they should do next or make resolving conflicts within the code and repository branches more difficult. We did manage to communicate better as we grew as a team. However, if the communication was there from the beginning, confusion and time wasting could have been avoided.

Rahma S Omar, M01044772 – Secretary

As the secretary, my role combined both organisational responsibilities as the secretary and technical contributions across the admin-side for the design, frontend, and backend development. Earlier in the project, I focused on researching the requirements and ensuring that the admin interface was consistent with the rest of the system.

As development progressed, I implemented admin-side frontend pages in Blazor. I built the admin login and dashboard pages as well as several other pages which were unique to the admin such as viewing the student and instructor databases and being able to remove inactive or idle accounts. Additionally, I decided to change the front end from the Figma design in the end as I realised that there were some inconsistencies, so I had to fix and account for that. I contributed to the backend by helping structure the admin controller and setting up the basic authentication flow and supporting features such as admin registration, password handling and login verification. We also researched pseudocode conventions and what should be included which helped the team understand how to put this into clear steps moving forward.

One of the challenges that I faced was balancing coding tasks with secretary responsibilities. Keeping track of meeting progress alongside the leader and ensuring that deadlines were communicated, required consistent organisation. Communication gaps within the group made it quite difficult to coordinate tasks efficiently at times, especially as different members were working on separate parts of the system.

If I were to approach this project again, I would encourage the team to establish clearer communication routines and set structured timelines. Overall, it is sufficient to say that this project was rewarding as it has strengthened my ability to manage both technical and organisational tasks and has improved my confidence greatly in contributing to full-stack development and working in a galvanising team environment.

Eloise R Yard, M01002265 - Developer

As a developer on this project, I focused primarily on building the system's architecture. I spent most of my time engineering the data management layer, where I built custom Node and List implementations for our Admin, Instructor, Learner and Lesson models, alongside the core domain models. It was important to build a solid, organised backbone for the application, as I knew efficiency at the data level would make the rest of the project run much smoother.

Once that structure was stable, I moved onto integrating the AI chatbot. I wanted to try and make sure this feature didn't mess with the logic we had already built. Balancing the technical heavy lifting of designing those data nodes with the modern AI implementation was a great challenge.

If I were to redo this project, I would try harder to establish more efficient communication within the whole group to minimise waiting around. Also, in moments where I was waiting for others to complete tasks, I would move on to other tasks rather than wasting time by not completing things that need to be completed.

Overall, this was a rewarding experience. I'm glad I got to see the full lifecycle, from raw data structures to the final, user-friendly features we originally designed and hoped for. I'm proud of what we delivered, we built something that is functional, and well structured.

Sihaam Muuse Ibraahim, M01036513 – Developer

As a developer, I contributed to both frontend and backend development, with a primary focus on learner functionality. I implemented the full C# backend for the learner side and developed the Blazor frontend for key pages including the home page, scheduling and FAQs. I also implemented search functionality across both learner and instructor features, contributed to instructor scheduling and developed the admin pages in C#.

A major part of my role was integrating the work of the team into a cohesive system. This required rewriting significant portions of both frontend and backend code to ensure consistency in structure, API usage and variable naming. I also converted multiple HTML and CSS pages into Blazor components to maintain a unified frontend, preventing design clashes and ensuring consistency throughout the application.

The main challenge was resolving inconsistencies between different implementations, which required extensive refactoring and debugging. This highlighted the importance of communication and shared coding standards within a team environment.

If I were to approach this project again, I would prioritise backend development earlier and establish clear coding standards from the start to reduce integration effort. Overall, this project strengthened my full-stack development skills and my ability to manage integration and deliver a cohesive system.

Ridaa A Mohamed, M00994466 – Tester

As the tester for this project, the main focus was ensuring that the frontend and backend worked together. The backend functioned consistently throughout different scenarios. While testing, I had encountered many issues relating to the database migrations, API responses and inconsistencies with the user flow. These helped me strengthen the stability of the system and ensured that each section behaved as intended.

One of the main challenges faced was making sure I maintained a synchronised backend environments with my team members. Issues such as outdated migrations, connection string different and conflicting database states lead to a delay, however resolving the, showed the importance of proper documentation and a consistent testing practice. By the end of this project, a smooth and reliable workflow which allowed the backend to run smoothly with the frontend worked on every team member machine.

Overall, the testing phase was important, as it demonstrated the importance of communication and structured debugging. To make this a better project in the future as a tester, I would begin with identifying the testing guidelines from the start and clearly identifying what needs to be implemented during the development phase, this would reduce the confusion with all the team members and ensure a smoother collaboration.